Contents

COMP 565 Fall 2026 Course Project **1**
RegExplain: Nucleotide-Level Attributions and Counterfactual Sequences for the Regulatory Code . . . 1
1. Background and motivation . . . 1
2. The prediction task . . . 4
3. Data . . . 5
4. Required methods . . . 6
5. Required experiments . . . 7
6. Making DiCE work on DNA . . . 8
7. Metrics . . . 10
8. Optional extensions . . . 13
9. Schedule (17 days) . . . 14
10. Deliverables . . . 14
11. Grading (100 points) . . . 15
12. Pitfalls to avoid . . . 15
13. References . . . 16

COMP 565 Fall 2026 Course Project

RegExplain: Nucleotide-Level Attributions and Counterfactual Sequences for the Regulatory Code

Instructor: Yue Li, School of Computer Science, McGill University **Groups:** 1 to 3 students **Duration:** under 3 weeks (a 17-day schedule is given in Section 9) **Report:** up to 8 pages including all figures, tables and references; 11 pt Arial or Times New Roman; 2 cm margins **Lectures you will use:** Lecture 7 and 8 (CNNs on DNA, in-silico mutagenesis, DeepLIFT), Lecture 9 (counterfactual explanation, DiCE, validity / proximity / sparsity / diversity, necessity and sufficiency)

Deadline: December 15, 2026

At a glance

Model	One small CNN predicting TF binding from 101 bp of DNA, trained by you
Data	Tier 0, a simulation you write (where the ground truth lives); Tier 1, one real TF, supplied as a prebuilt file
Required xAI	In-silico mutagenesis (ISM) and DeepLIFT
Required cAI	DiCE gradient, DiCE genetic, and a greedy baseline
Required experiments	E1 train, E2 attributions, E3 counterfactuals, E4 where the two meet
The question	Do the counterfactuals create the TF's motif, or just noise the model likes?
Everything else	Optional, listed in Section 8. Do none of it until E1 to E4 are finished.

---

1. Background and motivation

Transcription factors (TFs) read the genome through short sequence motifs. A single nucleotide substitution inside a motif can abolish binding and change gene expression, which is the mechanism behind a large fraction of non-coding disease-associated variants (Lecture 7, slides 3 to 5). Convolutional models in the style of DeepBind and Basset predict TF binding from sequence alone, but they learn the regulatory code as a black box: the first-layer filters only partially recapitulate known motifs, and the higher layers encode combinatorial logic that nobody has written down.

The course has now covered two complementary ways to open that box.

Feature attribution (xAI) answers "why". In-silico mutagenesis perturbs one nucleotide at a time and reads the change in the output. It is exact but costs $3n$ forward passes per sequence, and it is blind to cooperative effects because it only ever changes one position. DeepLIFT recovers per-nucleotide importance in a single backward pass by propagating differences from a reference. Lecture 8 showed precisely where the cheap methods break: plain gradients saturate, and DeepLIFT with the Rescale rule alone mis-assigns credit when two motifs act through an AND relation, which is the GATA1 plus TAL1 cooperativity case (slides 45 to 49).

Counterfactual explanation (cAI) answers "what to change". An attribution map tells you that positions 47 to 53 mattered. It does not tell you what to write there instead. A counterfactual is a nearby sequence that the same model scores differently, so it is a real model output rather than a surrogate's (Lecture 9, slide 3). In genomics this is not a metaphor: a counterfactual for an unbound sequence is a proposed edit, and unlike the credit-scoring examples from Lecture 9 it is testable in the lab.

The closing slide of Lecture 9 stated the gap this project targets. Across the works citing Mothilal et al. (2020), none apply DiCE to regulatory sequence; Section 1.1 says what the literature does and does not contain. Three obstacles make this interesting rather than routine:

1. **Structure.** A one-hot sequence is a long list of categorical features with 4 levels each, not a table of 10 mixed-type columns. Counterfactual search over it needs the categorical relaxation on Lecture 9 slide 10. Section 6 shows you exactly how to set this up.
2. **Plausibility.** Independently flipping nucleotides produces sequences no genome ever contained, so a "valid" counterfactual is very often just an adversarial example.
3. **Ground truth exists.** If a counterfactual for an unbound sequence is correct, it should have created a recognizable binding motif. JASPAR gives an independent, model-free check that the counterfactual is right for the right reason. No credit-scoring benchmark has this.

1.1 What the literature already contains, and what it does not

Read this before you write your introduction, because it is where most of your related-work section comes from and because it tells you what you can and cannot claim.

Already done. Minimal-edit optimization over one-hot DNA against a frozen pre-trained model is published and works. **Ledidi** (Schreiber et al., bioRxiv 2025) recasts editing as continuous optimization with an explicit term minimizing the number of edits, applies it to TF binding, accessibility and enhancer activity, and validates designed enhancers in cells. **Taskiran et al.** (Nature, 2024) show the phenomenon directly: roughly ten to fifteen single-nucleotide edits convert a random or inactive sequence into a cell-type-specific enhancer, and motif discovery shows activator sites being created along the way. **gReLU** packages Ledidi-style editing together with FIMO and JASPAR scanning, so the pipeline you are building exists as library code.

Not done. Those works are framed as *sequence design*, that is, make a good sequence. None of them is framed as *instance-level explanation*, that is, tell me what this classifier was keying on when it called this particular sequence unbound. None reports the Lecture 9 metric suite (validity, proximity, sparsity, diversity) on DNA. None produces several *diverse* alternative edit sets for the same input and scores them against each other. And motif creation is used in that literature as a biological sanity check on a design, not as a quantitative measure of whether an explanation was correct.

A terminology warning. "Counterfactual" already has a different meaning in regulatory genomics. A 2026 review from the Koo lab has a section called "Counterfactual probing through virtual perturbations", and what it means is in-silico mutagenesis and global importance analysis used to test necessity and sufficiency of a motif, not a constrained search for a minimal edit that flips a classifier. Say which sense you mean, once, early in your report.

The closest thing to this project in another biopolymer is Kłos et al. (arXiv, 2026), which does counterfactual explanation on protein sequence: minimal, plausible edits that flip a predictor, balancing validity, proximity and

plausibility. Worth reading for how they handle the plausibility term.

So: the mechanics are occupied, the framing and the evaluation protocol are not. This is a course project, not a paper, and you are not required to be novel. But if your results are interesting, this is where the contribution would sit, and your report should say so honestly rather than claiming more.

1.2 A short glossary, for students without a biology background

Most of you are computer scientists, not bioinformaticians. Nothing in this project needs a biology course, but it does use a dozen pieces of domain vocabulary. Here they are, in the order you will meet them.

Term	What it means for this project
bp	base pair, one letter of DNA. A 101 bp sequence is a string of length 101 over {A, C, G, T}.
Transcription factor (TF)	A protein that sticks to specific short patterns in DNA and switches nearby genes on or off. Our label is "does this TF stick here, yes or no".
Motif	The short pattern a TF prefers, typically 6 to 20 bp. It is fuzzy, not exact: CTCF tolerates several variants of its site.
PFM / PWM	The standard way to write a fuzzy motif down. A position frequency matrix is 4 by `w`, giving the probability of each base at each of the `w` motif positions. A position weight matrix is the same thing after a log-odds transform against a background. Think of it as a hand-curated convolution filter.
JASPAR	A public, curated database of PWMs, one or more per TF. We use it only as an answer key, never as model input.
ChIP-seq	The wet-lab experiment that tells us where a TF actually binds across the genome. It returns a list of genomic intervals.
Peak / summit	One interval where the TF was detected. The summit is the single bp of strongest signal, which is where we centre our 101 bp window.
narrowPeak	The tab-separated file format ENCODE distributes peaks in. Column 1 to 3 are chromosome, start, end; column 7 is signal; column 10 is the summit offset from the start.
IDR	A reproducibility filter applied across experimental replicates. "IDR thresholded peaks" means the peak list that survived it, which is the clean one you want.
hg38 / GRCh38	The current reference human genome assembly. A FASTA file of about 3.1 billion letters. You look up a sequence by chromosome, start and end.
GC content	The fraction of a sequence that is G or C. It varies a lot across the genome and is the single easiest shortcut for a sloppy classifier to learn, which is why Section 3 insists on GC-preserving negatives.
Dinucleotide shuffle	A random reordering of a sequence that keeps the count of every two-letter pair (AA, AC, AG, …) identical to the original. It therefore keeps GC and CpG content exactly, while destroying any real motif. This is how we manufacture honest negative examples.
FIMO	The tool that scans a PWM against a sequence and reports statistically significant matches. See below.
q-value	A p-value corrected for the fact that you ran many tests. `q < 0.05` means roughly "at most 5% of the matches I am calling significant are expected to be false".

1.3 What FIMO does, and why this project leans on it

FIMO ("Find Individual Motif Occurrences") answers one question: **given a motif written as a PWM, where in this sequence does that motif occur, and is the match better than chance?**

The algorithm is something you already understand under another name:

1. Slide the `4 x w` PWM along the sequence, one position at a time, on both the forward and the reverse strand.
2. At each position, score the window by summing the matrix entry for each observed base. That score is a log-odds ratio: how much more likely this window is under the motif model than under a plain background base-composition model. **This step is literally a 1D convolution with a fixed, curated filter**, which is exactly what the first layer of your CNN does with learned filters.
3. Convert each score to a p-value using the null distribution of scores under the background model.

4. Report every window whose p-value is below a threshold. Each reported window is a "hit".

(The original MEME-suite FIMO also reports q-values. The `memelite` reimplementation you will use deliberately does not, so this project thresholds on p-values. See Section 7.2.)

So the only thing FIMO adds over a convolution is calibrated statistics and a motif someone curated by hand rather than one your network invented.

Why it is the right evaluation tool here. Your CNN never sees JASPAR, and FIMO never sees your CNN. They are completely independent. That is what makes "the counterfactual created a FIMO hit for this TF's motif where the original had none" a meaningful external check rather than the model grading its own homework. It is the closest thing this project has to a ground-truth label on an explanation.

How to run it. Install `memelite` (`pip install memelite`), a pure-Python reimplementation. No MEME Suite installation required.

```
from memelite import fimo
# first argument: a MEME-format motif file (one matrix, or the whole JASPAR set)
# second argument: a FASTA file of sequences, or a numpy one-hot array
hits = fimo("CTCF_MA0139.meme", "my_counterfactual_sequences.fa")
```

The result tells you, per sequence, the position, strand, score and p-value of each match. Scan the originals and the counterfactuals separately and compare, which is the motif creation rate in Section 7.2.

One caution. FIMO finds motifs that are *present*, not motifs that are *functional*. A hit means the letters are there, not that a TF would bind in a living cell. Keep that distinction in your discussion section.

1.4 Learning objectives

- Train and validate a sequence classifier on real ChIP-seq derived data with honest negatives.
- Produce nucleotide-level attributions with ISM and DeepLIFT, and evaluate them against ground truth rather than by eye.
- Generate counterfactual sequences with both DiCE algorithms and quantify validity, sparsity and diversity.
- Judge whether a counterfactual is biologically meaningful or an adversarial artifact.
- Report a negative or ambiguous result clearly. Several comparisons may not have a guaranteed winner.

---

2. The prediction task

Train a binary classifier `f: {A,C,G,T}^L -> [0,1]` predicting whether a sequence of length `L` is bound by one chosen TF.

Use `L = 101`. This is deliberate. It keeps ISM cheap and, more importantly, it keeps the DiCE search space small enough that the gradient method finishes in minutes rather than hours. Do not use 500 bp or 1 kb windows.

Required model. A small CNN in PyTorch in the style of DeepBind and Basset (Lecture 7, slides 12 and 13):

```
one-hot (4 x 101) -> Conv1d(4 -> 64, kernel 15) -> ReLU -> MaxPool
                  -> Conv1d(64 -> 64, kernel 7)  -> ReLU -> GlobalMaxPool
                  -> Linear(64 -> 32) -> ReLU -> Linear(32 -> 1) -> sigmoid
```

Anything of roughly this size is fine. Aim for test AUROC in the 0.85 to 0.97 range. Training takes about two minutes on a free Colab T4 and about fifteen on a laptop CPU.

Required second model (Tier 0). Train the same architecture a second time on an independently simulated Tier 0 dataset, with a different random seed. This costs you two extra minutes and it is what makes the transferability

metric in Section 7.2 possible. A counterfactual that only fools the model that generated it is an adversarial example, not an explanation, and this is the cheapest way to find that out.

---

3. Data

Everything is public and downloadable without an account. **Tier 0 and Tier 1 are both required.**

Tier 0 (required): simulated sequences with known ground truth

You cannot benchmark an attribution method without knowing which nucleotides actually mattered, so start here. Reproduce a reduced version of the Lecture 8 simulation (slide 56):

- 101 bp background sequences, per-position ACGT probabilities 0.3, 0.2, 0.2, 0.3.
- Embed 1 to 3 instances of the **GATA1** motif at random non-overlapping positions in half the sequences; the other half gets none.
- Label 1 if any GATA1 instance is present, 0 otherwise. About 10,000 sequences total is plenty.
- **Record the embedded positions.** That mask is your ground truth for E2.

The motif must carry at least 8 bits of information content. This is not a detail. A plausible-looking but degenerate GATA1 matrix of 5.5 bits makes the task so hard that the Bayes-optimal classifier, a perfect PWM scan, only reaches AUROC 0.81, and you will spend days thinking your network is broken. Use this 7-position matrix (8.5 bits, consensus TGATAAG), rows in ACGT order:

```
T/W   0.10 0.05 0.05 0.80
G     0.03 0.03 0.91 0.03
A     0.91 0.03 0.03 0.03
T     0.03 0.03 0.03 0.91
A     0.91 0.03 0.03 0.03
A     0.88 0.04 0.04 0.04
G     0.05 0.05 0.85 0.05
```

Compute `sum(P * log2(P / background))` over the matrix and confirm you get about 8.5 before training anything. Then check the ceiling: score every 7-mer window with the log-odds matrix, take the per-sequence maximum, and compute its AUROC against the labels. If that ceiling is below 0.9, fix the matrix, not the model.

Sampling a motif instance from a JASPAR PFM and pasting it into a background string is about 30 lines of numpy, and writing it yourself is the recommended route. If you prefer a library, **simdna** (https://github.com/kundajelab/simdna) does this and ships the ENCODE and HOCOMOCO motif databases. The DeepLIFT genomics notebooks (https://github.com/kundajelab/deeplift/tree/master/examples/genomics) contain the original version of this simulation.

Tier 1 (required, but narrow): one real TF, from a prebuilt file

Tier 0 is where the clean evaluation lives, because it is the only place you know which nucleotides actually mattered. But it is a closed world: the matrix that generated the data, the signal your CNN learns, and the matrix FIMO scans with are all the same matrix. The whole argument in Section 1.1 for why genomics beats credit scoring is that JASPAR is an **independent, curated, model-free** answer key, and on Tier 0 it is not independent of anything. Tier 1 is what lets you write that sentence honestly.

You are given the data. Downloading a 1 GB genome and getting the ENCODE summit offset right is not what this project is teaching, and it is where most of the silent failures would happen. Your instructor ships three files:

File	Contents
`tier1_<TF>_<cell>.npz`	`seqs` int8 `(n, 101)` with A,C,G,T = 0,1,2,3; `y` float32; `chrom`; `split`; `neg_kind`
`tier1_<TF>_<cell>.meme`	the TF's JASPAR matrix, ready for FIMO
`tier1_<TF>_<cell>.json`	provenance: ENCODE accession, source URLs, counts, GC statistics

Load it with the supplied helper:

```
from tier1_loader import load_tier1, onehot
train, val, test = load_tier1("tier1_CTCF_K562.npz")
X, y = train                          # X is (n, 101) int8; onehot(X) gives (n, 4, 101)
```

The splits are fixed and held out by chromosome: chr1 and chr2 are test, chr8 and chr9 are validation, the rest is train. Do not re-split. Random splits leak through repeats and paralogous regions and inflate AUROC, and keeping the given split also makes every group's numbers comparable.

How it was built, because the construction is part of the lesson. Positives are 101 bp windows centred on the summit of the top 20,000 IDR-thresholded peaks (narrowPeak column 2 plus column 10). Negatives are **dinucleotide-preserving shuffles** of those same positives, one per positive. That preserves GC and CpG content exactly while destroying motif syntax, which is the standard construction and the one the Lecture 7 ChIP-seq schematic shows (slide 6). If the negatives had been uniform-random sequence instead, the model would reach AUROC 0.99 by counting GC and every counterfactual would be a GC-raising artifact. Check the GC columns in the `.json` and confirm for yourself that positives and negatives match; then say in your report why that matters.

The builder script `build_tier1.py` is included if you want to read it, rebuild with a different TF, or add the optional GC-matched genomic negative set. You are not expected to run it.

What you must do on Tier 1, and what you must not bother with.

- **Required:** retrain the same CNN architecture on the Tier 1 data (E1), then run E3 and report **two** metrics only: motif creation rate and transferability. That is the transfer check, and it is about an afternoon of work.
- **Not required:** the Tier 1 versions of E2's ground-truth metrics. Motif AUPRC against an embedded-motif mask does not exist on real peaks, because nobody knows which nucleotides mattered. Say so in your report rather than inventing a substitute.
- Expect lower AUROC than on Tier 0 and a lower motif creation rate. Real peaks contain co-factor motifs, repeats and indirect binding. That gap is a result, not a failure, and explaining it is worth more than any single number.

4. Required methods

Five methods, all installable with pip, all covered in lecture or in the Lecture 9 reading.

xAI

Method	Where	What to know
In-silico mutagenesis (ISM)	20 lines of your own, or `tangermeme.ism.saturation_mutagenesis`	Your reference method. For each position, substitute each of the other three bases and record the change in predicted probability. `3L = 303` forward passes per sequence. Batch them: at `L = 101` this is cheap, and DeepLIFT's speed advantage is only about 2x, not the orders of magnitude the asymptotic argument suggests.
DeepLIFT	**Captum**, `from captum.attr import DeepLift` (Lecture 8, slide 62)	One backward pass. Captum implements the Rescale rule. Requires a reference, see below.

Choosing the DeepLIFT reference. DeepLIFT is defined relative to a reference `x^0`, and on DNA an all-zero one-hot is not a sequence. Use the average of 10 dinucleotide-preserving shuffles of the input itself, which is the field standard. Also run it once with a uniform 0.25 reference and report how much your conclusions change. Do not assume the answer: on the Tier 0 simulation the two references agree almost perfectly (Spearman 0.95), so the honest finding may well be that the choice does not matter here. Say so if that is what you measure.

cAI

Method	Where	What to know
DiCE, gradient method	`pip install dice-ml`, `Dice(d, m, method="gradient")`, PyTorch backend	Optimizes the Lecture 9 slide 8 objective: validity plus proximity plus a DPP diversity term, with the sum-to-one penalty of slide 10. Needs the categorical relaxation and the setup in Section 6. The slowest of the three, about 30 s per query.
DiCE, genetic method	`Dice(d, m, method="genetic")`, sklearn backend	Model-agnostic search, so no relaxation is needed and the model is only ever queried. **Must be called with `proximity_weight=0.0`**: its proximity term is computed over continuous features only and raises `ValueError: cannot reshape array of size 0` when there are none. Leave `sparsity_weight` to do the minimality work.
Greedy directed evolution	about 20 lines of your own	Repeatedly make the single substitution that most increases the predicted probability, stop when the label flips. Costs about 0.03 s per query, so there is no excuse for leaving it out, and it is the sparsest method by a wide margin. It is also the one most likely to be finding an adversarial shortcut, which is exactly why you want it in the table.

Repository: https://github.com/interpretml/DiCE

---

5. Required experiments

E1. Train and characterize the model

Train on Tier 0 (two seeds, for the transferability metric) and on Tier 1 (one model is enough). Report AUROC, AUPRC and a calibration curve on the held-out split. Visualize the first-layer filters as sequence logos (`logomaker`) and state plainly how much of the known motif layer 1 actually recovered (Lecture 7, slides 11 and 13).

E2. ISM versus DeepLIFT

Run both on the same 200 test sequences. Report:

1. **Ground-truth recovery on Tier 0.** Per-nucleotide AUPRC of `|attribution|` against the embedded-motif mask. This is the cleanest number in the project.
2. **Agreement.** Per-sequence Spearman correlation between DeepLIFT and ISM scores, reported as a distribution and not a single number.
3. **Deletion curve.** Replace the top `k%` of positions by attribution with shuffled background and plot the drop in predicted probability against `k`, with a random-position control. Report the area under the degradation curve.
4. **Reference sensitivity.** The shuffled-reference versus uniform-reference comparison described in Section 4.
5. **One figure** of a bound sequence drawn as a logo scaled by attribution, with the motif's true location marked underneath. On Tier 0 you know that location exactly; on Tier 1 use the FIMO hit.

E3. DiCE gradient versus DiCE genetic versus greedy

Take test sequences the model scores as **negative** and generate counterfactuals it scores as **positive**. Use `k = 4` counterfactuals per input and at least 30 input sequences, all three methods on the same inputs with the same editable window.

Report the full Section 7.2 table. **Do the counterfactuals create the TF's motif, or do they just create noise the model happens to like?** Include at least one figure or text alignment showing an original sequence and its counterfactuals, with edited positions marked and any newly created FIMO hit annotated. Section 7.4 shows the format and what a correct answer looks like.

Expect validity to be 100% for all three methods. Validity is the easy metric here and on its own it says nothing; the comparison lives in sparsity, motif creation and transferability.

E4. Where xAI and cAI meet

Two short analyses, both cheap once E2 and E3 are done.

1. **Edit-attribution overlap.** Do the counterfactual's edited positions land on the high-attribution positions? Report precision at k of the edited set against the top-k positions by ISM and by DeepLIFT, with a random-position control.
2. **Necessity and sufficiency**, adapted from Lecture 9 slide 26 from features to nucleotide positions:
   - **Necessity** of a position set `S`: allow only `S` to change and measure the fraction of inputs that still yield a valid counterfactual.
   - **Sufficiency** of `S`: freeze `S` at its original values, let everything else move, and measure how many counterfactuals the freeze destroys.

   Run this for the top 5 and top 10 positions by each attribution method. Reproduce the shape of the Vlontzou et al. figure. If freezing the top-10 ISM or DeepLIFT positions does not stop DiCE from flipping the label, say so and explain what that implies about the model.

Two warnings, both from a reference run of this project.

- **The overlap metric is meaningless for dense counterfactuals.** A method that edits 26 of 41 editable positions hits the top-k by saturation and scores the same as its own random control. Always report the control next to the precision. Only the sparse methods produce an interpretable number here.
- **Sufficiency may be 0% in every setting.** Freezing the top positions does not prevent a flip, because the remaining positions in the window can always produce one. That is a property of the task, not a bug in your code. Report it and interpret it: the top positions are sufficient but not necessary, which is the same shape as the Vlontzou et al. result.

---

6. Making DiCE work on DNA

This is the only genuinely fiddly step in the project. Follow this recipe exactly; every line in it exists because something went wrong without it in a reference run.

The encoding. Treat each position as one categorical feature with levels {A, C, G, T}. Build a pandas DataFrame with one column per position, values as single-character strings, plus the label column:

```
import pandas as pd, dice_ml

# ZERO-PAD THE NAMES. This is not cosmetic. See the warning below.
cols = [f"p{i:03d}" for i in range(101)]        # p000 ... p100
```

```
df = pd.DataFrame([list(s) for s in sequences], columns=cols)
df["bound"] = labels
d = dice_ml.Data(dataframe=df, continuous_features=[], outcome_name="bound")
```

> **The zero padding is load-bearing.** DiCE groups the one-hot columns belonging to one categorical feature by **string prefix match on the feature name**. With columns named `p0 ... p100`, the feature `p1` also captures `p10_A ... p19_T` and `p100_A ... p100_T`. In a reference run this produced groups of 48 columns instead of 4, which made the sum-to-one penalty charge `(12 - 1)^2` per group: a regularization loss of 27,806 against a hinge loss of 3.7. It also made `features_to_vary` resolve to an empty index set, so DiCE zeroed every gradient and nothing moved for 5,000 iterations. The error message you get is "perhaps try with different values of proximity (or diversity) weights or learning rate", which sends you tuning hyperparameters for a naming bug. Do not lose a week to this.

DiCE one-hot encodes these columns internally and, for the gradient method, relaxes them to continuous values in [0,1] with a penalty forcing the four levels at each position to sum to one, then takes the arg max at the end. This is exactly the mechanism on Lecture 9 slide 10, now applied to DNA.

Wrapping your PyTorch model. The gradient method hands your model the *transformed* frame: 404 one-hot columns in DiCE's own order, not your `(N, 4, 101)` tensor. So you need a thin `nn.Module` that permutes and reshapes. Read the column order off the transformer rather than assuming it:

```
from dice_ml.utils.helpers import DataTransfomer

t = DataTransfomer("ohe-min-max"); t.feed_data_params(d); t.initialize_transform_func()
enc = [c for c in t.transform(df.iloc[[0]]).columns if c != "bound"]   # 404 names

class FlatWrapper(nn.Module):
    def __init__(self, cnn, enc):
        super().__init__(); self.cnn = cnn
        idx = {}
        for j, c in enumerate(enc):                 # "p053_G" -> (53, 2)
            pos, base = c.rsplit("_", 1)
            idx[(int(pos[1:]), "ACGT".index(base))] = j
        perm = np.array([[idx[(p, b)] for p in range(101)] for b in range(4)])
        self.register_buffer("perm", torch.tensor(perm.ravel()))

    def forward(self, x):
        x = x.float()
        squeeze = (x.dim() == 1)          # DiCE passes a flat vector in some paths
        if squeeze: x = x.unsqueeze(0)
        z = x[:, self.perm].reshape(-1, 4, 101)
        out = torch.sigmoid(self.cnn(z)).reshape(-1, 1)
        return out[0] if squeeze else out

m = dice_ml.Model(model=FlatWrapper(net, enc), backend="PYT", func="ohe-min-max")
```

Two traps in that block. `func="ohe-min-max"` is required; with `func=None` the transformer is a no-op, DiCE hands your model 101 raw columns instead of 404, and you get `IndexError: index 104 is out of bounds for dimension 0 with size 101` from inside your own code. And the `dim() == 1` branch is required because DiCE calls the model with a flat vector in some code paths and a batch in others.

Keep the search space small. Restrict the editable window with `features_to_vary` to the central 41 bp, which is where a motif in a summit-centred window will be:

```
mutable = [f"p{i:03d}" for i in range(30, 71)]
exp = dice_ml.Dice(d, m, method="gradient")
cfs = exp.generate_counterfactuals(query_df, total_CFs=4, desired_class=1,
                                   features_to_vary=mutable)
```

Report that you did this. It is a modelling choice, not a detail.

The genetic method takes the raw categorical frame instead, so wrap the network in a plain object exposing `predict_proba` and use `backend="sklearn"`. Nothing needs to be differentiable. Call it with `proximity_weight=0.0` for the reason given in Section 4.

Expected cost, measured on two CPU cores: gradient method about 28 s per query with 41 mutable positions and `k = 4`, so 30 queries is about 14 minutes; genetic method about 3 s per query; greedy about 0.03 s. If the gradient method is too slow, cut to 21 mutable positions before you cut the number of inputs.

When DiCE returns nothing, instrument before you tune. Print the loss *components*, not just the total:

```
from dice_ml.explainer_interfaces.dice_pytorch import DicePyTorch
orig = DicePyTorch.compute_loss
def probe(self):
    out = orig(self)
    print(float(self.loss), float(self.yloss), float(self.regularization_loss))
    return out          # must return it: the caller uses the value
DicePyTorch.compute_loss = probe
```

If the regularization term is three orders of magnitude larger than the y-loss, your column names are colliding. If every number is frozen across hundreds of iterations, your gradients are being zeroed. One instrumented run localizes either problem; nine hyperparameter sweeps will not. This is the transferable research skill in this project, and it is worth a paragraph in your report.

---

7. Metrics

Report distributions (median and interquartile range) for everything that is per-sequence.

7.1 Attribution metrics (E2)

Metric	Definition
Motif AUPRC	Per-nucleotide precision-recall AUC of |attribution| against the ground-truth motif mask (Tier 0 only)
ISM agreement	Per-sequence Spearman correlation between DeepLIFT and ISM
Deletion AUC	Area under the curve of predicted probability against fraction of top-attribution positions replaced by shuffled background; lower is better. Report the random-position control alongside.
Reference sensitivity	Correlation between attributions computed with a shuffled reference and with a uniform 0.25 reference

7.2 Counterfactual metrics (E3)

The first four are the Lecture 9 slide 14 set, translated to sequences.

Metric	Definition
% Valid CFs	Fraction of the `k` returned counterfactuals that are unique and in the target class

Metric	Definition
Sparsity	Hamming distance to the original, that is the number of substituted nucleotides. Report the median and the distribution, and the normalized `d_H / L`.
Proximity	`1 - d_H / L`, plus the change in GC content
Diversity	Mean pairwise Hamming distance among the `k` counterfactuals for the same input
Motif creation rate (the ground-truth metric)	Fraction of valid counterfactuals in which FIMO reports a significant hit for the TF's motif that was absent in the original. Use a **p-value** threshold (1e-4 is `memelite`'s default); `memelite` deliberately does not compute q-values, so if you want FDR control you must apply Benjamini-Hochberg yourself and say so. Also report the drop in the best-hit p-value. Install with `pip install memelite`; it gives you FIMO in pure Python with no MEME suite install. Section 1.3 explains what FIMO does.
Transferability	Fraction of counterfactuals also classified positive by the second-seed model from Section 2. Low transferability means you generated adversarial examples. **The converse does not hold**, and this trips people up: in a reference run all three methods transferred at 73% to 86% while their motif creation rates ranged from 4% to 57%. Two models trained on independent data can share the same shortcut. Transferability and motif creation are different questions, and you need both.

7.3 What a good result looks like

You are not expected to find that one method wins on everything. Lecture 9 slide 5 says these criteria conflict by construction. What you should be able to do is say *which* trade-off each method settled and whether the result is biologically meaningful.

The claim to test, and the one that carries the report, is this: **sparsity and correctness are not the same axis.** The sparsest counterfactual is not the most correct one. A method can flip the label with two nucleotides and have created nothing a biologist would recognize, while a method that uses seven nucleotides writes the actual motif. Design your tables so a reader can see that, and state plainly which of your methods is finding biology and which is finding adversarial examples.

A report claiming 100% validity without the transferability and motif-creation checks has not answered the question.

7.4 Expected results on Tier 0 (reference run)

These numbers come from one complete run of E1 to E4 on the Tier 0 simulation with the matrix in Section 3, 30 query sequences, `k = 4`, and a 41 bp editable window. **They are a sanity reference, not a target.** Your seeds, your matrix and your window will move them. Use them to tell a bug from a result: if your motif AUPRC is 0.15 or your DiCE run returns nothing, something is broken; if your motif creation rate is 45% instead of 57%, that is just a different run.

E1. Test AUROC 0.957 and 0.950 for the two seeds, AUPRC 0.962 and 0.957. Training takes under 10 s on two CPU cores. If you are below 0.85, check the motif's information content before touching the architecture.

E2.

	ISM	DeepLIFT (shuffled ref)	random
Motif AUPRC	0.47	**0.68**	0.14
Deletion AUC (lower better)	**0.35**	0.40	0.62
Spearman, DeepLIFT vs ISM	-	0.57 (IQR 0.47 to 0.65)	-
Seconds per sequence	0.014	0.007	-

Two things here are worth your discussion section. **DeepLIFT beats ISM on ground-truth motif recovery**, which is the saturation problem of Lecture 8 seen from the attribution side: with two or three motif copies present, knocking out one nucleotide of one copy barely moves the prediction, so ISM scores it near zero. And **ISM wins the deletion**

curve for a circular reason: that metric is built from the same single-position replacement operation ISM optimizes. Noticing this is worth more marks than the numbers themselves.

E3.

Metric	DiCE gradient	DiCE genetic	Greedy
% valid CFs	100	100	100
sparsity, median edits	7	26	**2**
fraction of the 41 bp window rewritten	18%	63%	5%
diversity, mean pairwise Hamming	11.7	**27.4**	2.5
motif creation rate	**57%**	26%	4%
transferability to the second model	86%	83%	73%
seconds per query	28.4	3.1	0.03

The ground-truth motif in that run was 7 bp wide, and the gradient method's median counterfactual used 7 edits. It is not finding the cheapest way to fool the model; it is writing the motif.

E4. Edit-attribution precision at 5 against ISM: 0.48 for the gradient method (random control 0.19), 0.65 for the genetic method (control 0.61, that is, no signal at all), 0.83 for greedy (control 0.15). Necessity of the top-5 ISM positions: 93%. Sufficiency: 0% in every setting.

What a correct counterfactual looks like Ground-truth motif consensus **TGATAAG**. `^` marks an edited position, `[-----]` marks a FIMO hit present in the counterfactual and absent in the original. Positions 28 to 74 shown.

DiCE gradient, 6 edits, writes one clean motif:

```
query #0   edits=6   P(bound): 0.053 -> 0.981   second model: 0.967
pos        30        40        50        60        70
original AATAAATTAATATCTCATTGAGCGACGCTAAGAAAGGAAAGCGTTC
counterf AATGTATTAATATCTCAATGAGCGATGATAAGAAACGAAAGCGTTC
edits      ^^            ^       ^ ^       ^
FIMO hit                         [------]
         created ATGATAAG at 53-60, p = 6.1e-05
```

DiCE gradient, 8 edits, writes *two* copies, which is what the training distribution contains:

```
query #2   edits=8   P(bound): 0.094 -> 0.999   second model: 0.998
pos        30        40        50        60        70
original TAGTGATATTATGTCATATGTTGAAGACATAAGCATATAAAATTTT
counterf TATTGATAAGATGTGATAAGTTGAAGTCATAAGCATATAGCATTTT
edits      ^     ^^    ^   ^       ^            ^^
FIMO hit   [------]  [------]
         created TTGATAAG at 31-38 and GTGATAAG at 41-48
```

DiCE genetic reaches the same place but buries it: 23 edits for one motif, and the other 21 are doing nothing a biologist could act on.

```
query #0   edits=23   P(bound): 0.053 -> 0.995
original AATAAATTAATATCTCATTGAGCGACGCTAAGAAAGGAAAGCGTTC
counterf AATACAATGATAATTCCTTTACTGATAAGTTTACTCGATCGAATTC
edits        ^ ^ ^   ^^  ^  ^ ^^  ^^^^^^^ ^^^  ^^ ^^
FIMO hit                     [------]
```

Greedy, when it does create the motif, is strikingly economical. Two substitutions turn `CGCTAAG` into `TGATAAG`:

```
query #0   edits=2   P(bound): 0.053 -> 0.972   second model: 0.959
original AATAAATTAATATCTCATTGAGCGACGCTAAGAAAGGAAAGCGTTC
counterf AATAAATTAATATCTCATTGAGCGATGATAAGAAAGGAAAGCGTTC
edits                             ^ ^
FIMO hit                         [------]
```

But this happened in only 4% of its counterfactuals. The other 96% flipped the label without building anything recognizable. Producing an alignment like these for your own counterfactuals, including the failures, is part of E3.

A caution you should carry into your discussion: notice that the gradient counterfactuals also carry edits far from the created hit (query #0 has two at positions 31 to 32, while the motif is at 53 to 60). None of these methods produces a minimal explanation. They produce a much-less-padded one.

---

8. Optional extensions

Do none of these until E1 to E4 are complete and written up. One is plenty, and a project that does E1 to E4 well scores higher than one that does five extensions badly.

More xAI methods. Gradient times input and Integrated Gradients (both in Captum), DeepSHAP (`captum.attr.DeepLiftShap` or `shap.DeepExplainer`), and DeepLIFT with the **RevealCancel** rule, which is only in the original `deeplift` package (https://github.com/kundajelab/deeplift) and is the rule that fixes the AND-relation failure. Running TF-MoDISco-lite (`pip install modisco-lite`) on your attribution maps to discover motifs de novo is a natural follow-on.

More cAI methods.

- **Ledidi** (https://github.com/jmschrei/ledidi, `pip install ledidi`), a gradient-based sequence editor purpose-built for differentiable PyTorch models taking one-hot `(N, 4, L)` input. As Section 1.1 explains, this is the published state of the art for the editing step, and it is the single most informative comparison you can add. If you have time for exactly one extension, make it this one. It is the same idea as DiCE's gradient method but not the same algorithm, and the two differences are what you are actually measuring:
  - **Sparsity.** Ledidi's loss is output loss plus a weight `l` times the count of edits, so minimality is optimized directly. DiCE's gradient objective has no sparsity term, because the L0 count is non-convex; it restores features greedily *after* optimization while the predicted class still holds (Lecture 9, slide 11).
  - **Discreteness.** DiCE relaxes the one-hot columns to continuous values with a sum-to-one penalty and takes the arg max only at the end, so throughout the search the model is scored on a soft mixture that is not a real sequence and the final arg max can lose validity. Ledidi samples a hard one-hot sequence from a Gumbel-softmax at every step and passes the gradient through the relaxation, so the model always sees a legal sequence.
  - One thing DiCE has and Ledidi does not: a diversity term. Ledidi's batch comes from a single weight matrix, so its edits are correlated by construction; you get alternative edit sets by re-running with a different seed. Predict before you run which method wins on which metric, then report whether you were right.
- **Greedy directed evolution**: repeatedly make the single substitution that most increases the score. Simple, fast, and a surprisingly strong sparsity baseline.
- **Permute Attack** style genetic search reimplemented from Lecture 9 slides 23 and 24, with the explicit `rho_0 * L0` sparsity term.
- **Wachter baseline** via `alibi` (https://github.com/SeldonIO/alibi), the single-counterfactual no-diversity version from Lecture 9 slide 4.

Harder data or constraints. A second negative type, GC-matched genomic background instead of dinucleotide

shuffles, and a comparison of what it does to the counterfactuals. If your Tier 1 file was built with `--gc-matched` it already contains that set; `load_tier1(..., negatives="gcmatch")` switches to it, and `"both"` uses both. An edit budget sweep: cap substitutions at 3, 5 and 10 and plot validity against budget. A frozen central window, which is DiCE's actionability constraint with a biological meaning.

Cooperativity. Add TAL1 to the Tier 0 simulation with a multi-task head (both present / only GATA1 / only TAL1 / neither, as in Lecture 8 slide 56), then ask which motif the counterfactual chooses to destroy and whether DeepLIFT Rescale and RevealCancel agree. This is the Lecture 8 slide 45 failure seen from the counterfactual side.

Plausibility scoring. KL divergence between the counterfactual's dinucleotide frequencies and those of real bound sequences, or log-likelihood under a 3rd-order Markov background model fit on the genome.

---

9. Schedule (17 days)

Days	Work
1 to 3	Form group. Build the Tier 0 simulation, verify the motif's information content and the PWM-scan ceiling, train both seeds. Finish E1 on Tier 0. **Freeze the simulation on day 3.**
4 to 7	E2. ISM, DeepLIFT, deletion curve with its random control, reference comparison. One clean figure.
8 to 12	E3 on Tier 0. Day 8 is for the DiCE encoding (Section 6); if it is not working by the end of day 8, instrument the loss components rather than tuning. Days 9 to 11 generate and score counterfactuals for all three methods. Day 12 is E4, which is quick once E3 exists.
13 to 14	Tier 1. Load the prebuilt file, retrain the CNN, re-run E3 and report motif creation and transferability. Compare against Tier 0 and explain the gap.
15 to 17	Tables, figures, writing, and a reproducibility pass on the repository.

The schedule assumes you are **given** the Tier 1 file. If you decide to rebuild it yourself, add three days and do not say you were not warned.

If you fall behind, cut in this order: the optional extensions first, then the Tier 1 transfer check (a complete project on Tier 0 alone still earns a good grade, it just cannot claim the independent-answer-key argument), then the number of input sequences in E3. **Never cut the transferability or motif-creation metrics.** They are what make the counterfactual results interpretable.

---

10. Deliverables

1. **Report**, PDF, up to 8 pages including all figures, tables and references. 11 pt Arial or Times New Roman, 2 cm margins on all sides. Suggested structure: Introduction and motivation; Data and model; Methods; Results; Discussion and limitations; Contributions; References. Spend the space on results, not on restating the lectures.
2. **Code repository** (GitHub or a zip) with a README giving the exact commands to reproduce every table and figure, a `requirements.txt`, and the random seeds used. The Tier 0 simulation must be generated by a script in the repository, not pasted in as a data file.
3. **Contribution statement**, one short paragraph naming what each group member did. Required for groups of 2 and 3.

Packages you will need: `torch`, `captum`, `dice-ml`, `memelite`, `logomaker`, `scikit-learn`, `pandas`, `numpy`. Optional: `tangermeme`, `deeplift`, `shap`, `modisco-lite`, `lediDi`, `alibi`.

You are given `tier1_<TF>_<cell>.npz`, its `.meme` and `.json`, `tier1_loader.py`, and `build_tier1.py` for reference. Do not commit the `.npz` to your repository; reference it by name.

Pin your versions in `requirements.txt`. The reference run used torch 2.14.1, captum 0.9.0, dice-ml 0.12, memelite 0.5.0, numpy 2.4.4, pandas 3.0.2, scikit-learn 1.8.0. The dice-ml behaviour described in Sections 4 and 6 is version-specific; a different release may move or fix those rough edges.

---

11. Grading (100 points)

Component	Points	What earns full marks
Data and model	20	A Tier 0 simulation whose motif information content and PWM-scan ceiling you verified, a working second-seed model, the supplied Tier 1 splits used unchanged, and an explanation of why the negatives are dinucleotide shuffles rather than random sequence
E2 attributions	25	ISM and DeepLIFT compared properly, ground-truth AUPRC on Tier 0, deletion curve with a random control, reference sensitivity
E3 counterfactuals	25	All three methods on Tier 0 with the full Section 7.2 table, plus the Tier 1 transfer check (motif creation and transferability only)
E4 overlap, necessity and sufficiency	15	Correct adaptation of the definitions, results across x/cAI methods, a clear statement of what it implies
Report quality and critical discussion	10	Honest limitations, recognition of the validity-versus-sparsity trade-off, no overclaiming
Reproducibility	5	The repository runs from a clean environment
Optional extension	up to +5 bonus	One extension from Section 8, done properly

Groups of 1 may use 30 input sequences in E3 rather than more, and may skip the Tier 1 transfer check if they say so and explain what claim they therefore cannot make. Groups of 3 are expected to complete one Section 8 extension.

---

12. Pitfalls to avoid

- **Uniform-random negatives.** If you build your own negatives anywhere, make them dinucleotide shuffles. With random sequence the model hits AUROC 0.99 by counting GC and every counterfactual is a GC-raising artifact.
- **Reporting a motif AUPRC on Tier 1.** There is no ground-truth mask on real peaks. That metric is Tier 0 only.
- **Re-splitting the Tier 1 data.** The splits are chromosome-held-out and fixed. A random re-split leaks through repeats and paralogs and will inflate your AUROC.
- **All-zero references for DeepLIFT.** Use dinucleotide shuffles and say which you used.
- **Long sequences.** 101 bp is specified for a reason. At 500 bp the DiCE gradient method may be too slow.
- **Reporting validity alone.** Validity is the easy metric and is close to meaningless without sparsity, motif creation and transferability alongside it.
- **Cherry-picked example figures.** Show a randomly selected example, or show the distribution.
- **Treating JASPAR as a label for training.** JASPAR is your independent evaluation. If the model or the counterfactual generator ever sees it, the motif creation rate becomes circular.
- **Tuning hyperparameters before instrumenting.** When a method returns nothing, print the loss components first. See the end of Section 6.
- **Judging a counterfactual method by validity or sparsity alone.** Both are easy to win and neither tells you whether the edit means anything.
- **Claiming a biological discovery.** The right claim is “the model’s decision is consistent with the known motif”, not “we found a new regulatory element”.

---

13. References

1. Alipanahi B, Delong A, Weirauch MT, Frey BJ. Predicting the sequence specificities of DNA- and RNA-binding proteins by deep learning. *Nature Biotechnology* 33(8):831-838, 2015.
2. Kelley DR, Snoek J, Rinn JL. Basset: learning the regulatory code of the accessible genome with deep convolutional neural networks. *Genome Research* 26(7):990-999, 2016.
3. Zhou J, Troyanskaya OG. Predicting effects of noncoding variants with deep learning-based sequence model. *Nature Methods* 12(10):931-934, 2015.
4. Shrikumar A, Greenside P, Kundaje A. Learning important features through propagating activation differences. *ICML*, 3145-3153, 2017.
5. Lundberg SM, Lee SI. A unified approach to interpreting model predictions. *NeurIPS* 30, 4765-4774, 2017.
6. Adebayo J, Gilmer J, Muelly M, Goodfellow I, Hardt M, Kim B. Sanity checks for saliency maps. *NeurIPS*, 2018.
7. Wachter S, Mittelstadt B, Russell C. Counterfactual explanations without opening the black box: automated decisions and the GDPR. *Harvard Journal of Law and Technology* 31(2):841-887, 2017.
8. Mothilal RK, Sharma A, Tan C. Explaining machine learning classifiers through diverse counterfactual explanations. *FAT* 2020*, 607-617.
9. Guidotti R. Counterfactual explanations and how to find them: literature review and benchmarking. *Data Mining and Knowledge Discovery* 38:2770-2824, 2024.
10. Hashemi M, Fathi A. PermuteAttack: counterfactual explanation of machine learning credit scorecards. *arXiv:2008.10138*, 2020.
11. Vlontzou ME, Athanasiou M, Dalakleidi KV, Skampardoni I, Davatzikos C, Nikita K. A comprehensive interpretable machine learning framework for mild cognitive impairment and Alzheimer's disease diagnosis. *Scientific Reports* 15:8410, 2025.
12. Kokhlikyan N, et al. Captum: a unified and generic model interpretability library for PyTorch. *arXiv:2009.07896*, 2020.
13. JASPAR: the open-access database of transcription factor binding profiles. Release 11 (2026). https://jaspar.elixir.no
14. ENCODE Project Consortium. An integrated encyclopedia of DNA elements in the human genome. *Nature* 489:57-74, 2012.
15. Schreiber J, Lorbeer FK, Heinzl M, Reiter F, Rafanel B, Lu YY, Stark A, Noble WS. Programmatic design and editing of cis-regulatory elements. *bioRxiv* 2025.04.22.650035, 2025. https://www.biorxiv.org/content/10.1101/2025.04.22.650035
16. Taskiran II, Spanier KI, et al. (Aerts S). Cell-type-directed design of synthetic enhancers. *Nature* 626:212-220, 2024.
17. Nagai M, Murphy AE, Rizzo K, Koo PK. Toward interpretable and generalizable AI in regulatory genomics. *arXiv:2602.01230*, 2026.
18. Linder J, Bogard N, Rosenberg AB, Seelig G. A generative neural network for maximizing fitness and diversity of synthetic DNA and protein sequences. *Cell Systems* 11(1):49-62, 2020.
19. Kłos W, Bender S, Kades L. Protein counterfactuals via diffusion-guided latent optimization. *arXiv:2603.10811*, 2026.
20. Lal A, Gunsalus L, Nair S, Biancalani T, Eraslan G. gReLU: a comprehensive framework for DNA sequence modeling and design. *bioRxiv* 2024.09.18.613778, 2024. https://github.com/Genentech/gReLU